

WEEK 5 TASK

Name: Rohit Balasaheb Kokare

ID: CT_CSI_DE_1172

College: Sanjivani College of Engineering

Objective:

Objective: Understand Spark fundamentals and perform data cleaning, transformation, and aggregation using DataFrames. Steps: Understand limitations of MapReduce and advantages of Spark (in-memory processing, speed). Learn Spark DataFrame concepts and immutability. Perform data cleaning operations (remove duplicates, handle null values). Apply filtering conditions on datasets (age range, category, region). Use aggregation functions (count, sum, avg, min, max). Group data using groupBy and apply conditions on aggregated results. Understand wide transformations and shuffle operations. Modify schema (casting, renaming columns). Handle inconsistent data (nulls, empty values, schema issues). Build a complete data processing pipeline combining cleaning and aggregation. Output: Spark code (PySpark/Scala) + query results + brief insights on data processing and transformations.

Resources:

<https://celebaltech.sharepoint.com/:w:/s/Celebal-LMS/IQCA80CDhtBJTleBBsbuwgAGAQr7dZ2qpz4wIYC2cjFJXM?e=cfZGCB>

Step 1: Import Required Libraries

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

Step 2: Create Spark Session:

```
spark = SparkSession.builder.appName("Week5Assignment").getOrCreate()
```

Step 2: Create Spark Session:

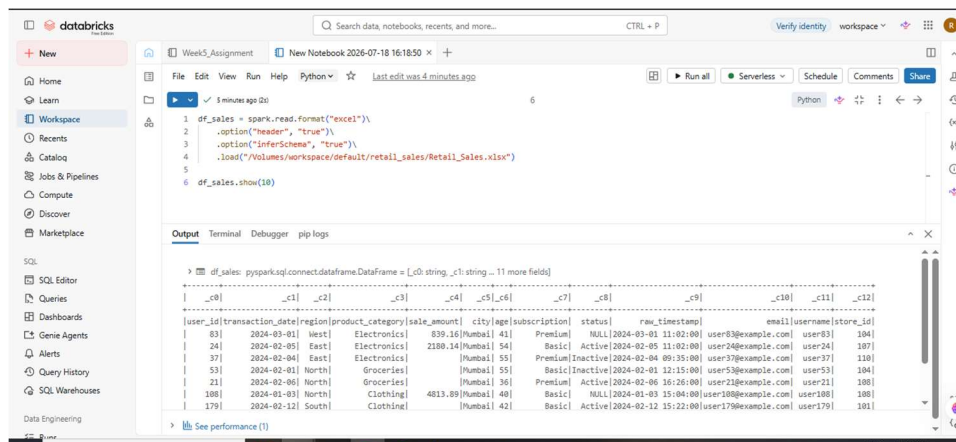
```
spark = SparkSession.builder.appName("Week5Assignment").getOrCreate()
```

Step 3: Load Dataset

```
df_sales = spark.read.format("excel")\
    .option("header", "true")\
    .option("inferSchema", "true")\
    .load("/Volumes/workspace/default/retail_sales/Retail_Sales.xlsx")

df_sales.show(10)
```

Output:



Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer

- Writes intermediate results to disk after every stage.
- High disk I/O makes processing slow.
- Poor performance for iterative machine learning algorithms.
- Difficult to write complex multi-stage workflows.
- High latency for interactive analytics.
- Limited APIs compared to Spark.

Advantages of Spark

- In-memory processing
- Faster execution
- Rich APIs (SQL, DataFrames, MLlib)
- Supports streaming and graph processing

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

Spark stores intermediate data in **RAM (memory)** instead of writing it to disk after each operation.

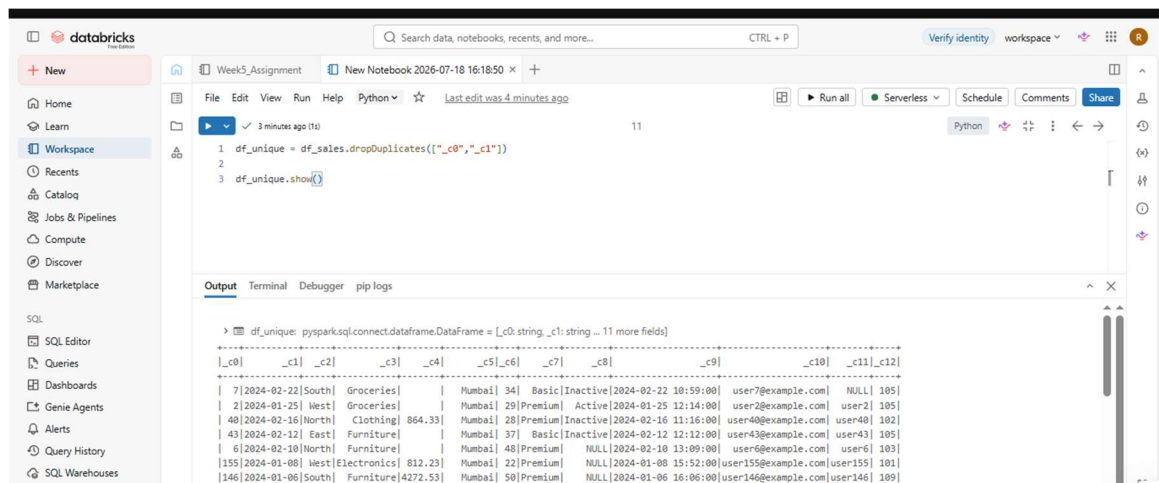
For iterative algorithms like K-Means or Logistic Regression, Spark reads the data once, caches it in memory, and reuses it for multiple iterations. This greatly reduces disk I/O and speeds up execution compared to MapReduce.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

```
df_unique = df_sales.dropDuplicates(["_c0", "_c1"])

df_unique.show()
```

output:



The screenshot displays a Databricks workspace with a notebook titled 'New Notebook 2026-07-18 16:18:50'. The code in the notebook is as follows:

```
1 df_unique = df_sales.dropDuplicates(["_c0", "_c1"])
2
3 df_unique.show()
```

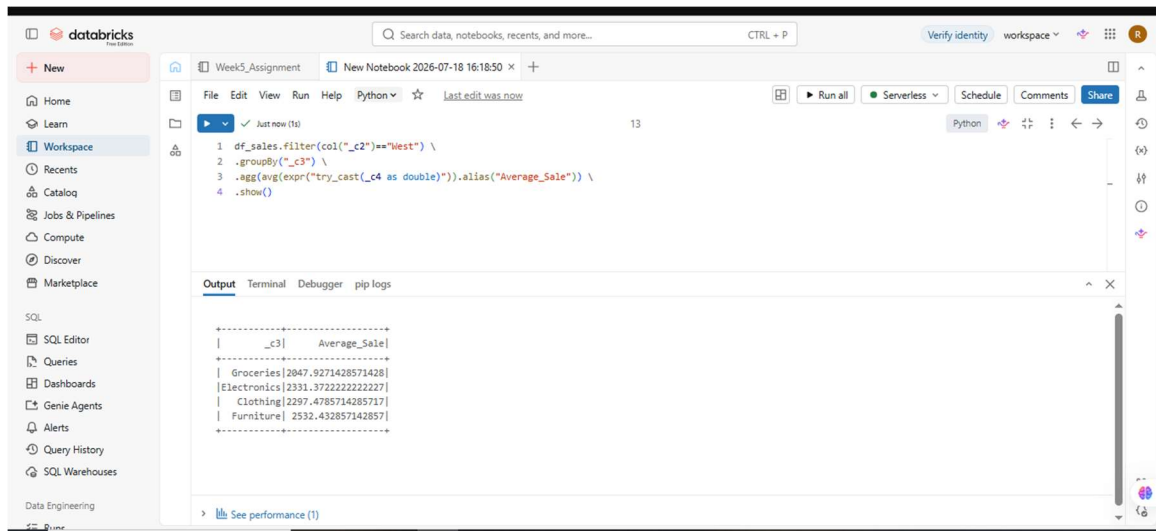
The output of the code is displayed in the 'Output' tab, showing a DataFrame with 11 rows and 12 columns. The columns are labeled from _c0 to _c12. The data includes transaction details such as date, region, product category, and user information.

_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7	_c8	_c9	_c10	_c11	_c12
7	2024-02-22	South	Groceries		Mumbai	34	Basic	Inactive	2024-02-22 10:59:00	user7@example.com	NULL	105
2	2024-01-25	West	Groceries		Mumbai	29	Premium	Active	2024-01-25 12:14:00	user2@example.com	user2	105
40	2024-02-16	North	Clothing	864.33	Mumbai	28	Premium	Inactive	2024-02-16 11:16:00	user40@example.com	user40	102
43	2024-02-12	East	Furniture		Mumbai	37	Basic	Inactive	2024-02-12 12:12:00	user43@example.com	user43	105
6	2024-02-10	North	Furniture		Mumbai	48	Premium		2024-02-10 13:09:00	user6@example.com	user6	103
155	2024-01-08	West	Electronics	812.23	Mumbai	22	Premium		2024-01-08 15:52:00	user155@example.com	user155	101
146	2024-01-06	South	Furniture	4272.53	Mumbai	50	Premium		2024-01-06 16:06:00	user146@example.com	user146	109

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

```
df_sales.filter(col("_c2")=="West") \
.groupBy("_c3") \
.agg(avg(expr("try_cast(_c4 as double)")).alias("Average_Sale")) \
.show()
```

Output:



Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

- `na.drop()` removes rows containing null values.
- `na.fill()` replaces null values with specified values.

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Search data, notebooks, recent, and more...

CTRL + P

Verify identity

workspace

Week3_Assignment

New Notebook 2024-07-18 16:18:50

+

File Edit View Run Help Python

last edit was 2 minutes ago

Run all

Serverless

Schedule

Comments

Share

1 minute ago (1)

15

Python

```
1 df_sales.na.fill({"_c8": "Unknown"}).show()
```

Output

Terminal

Debugger

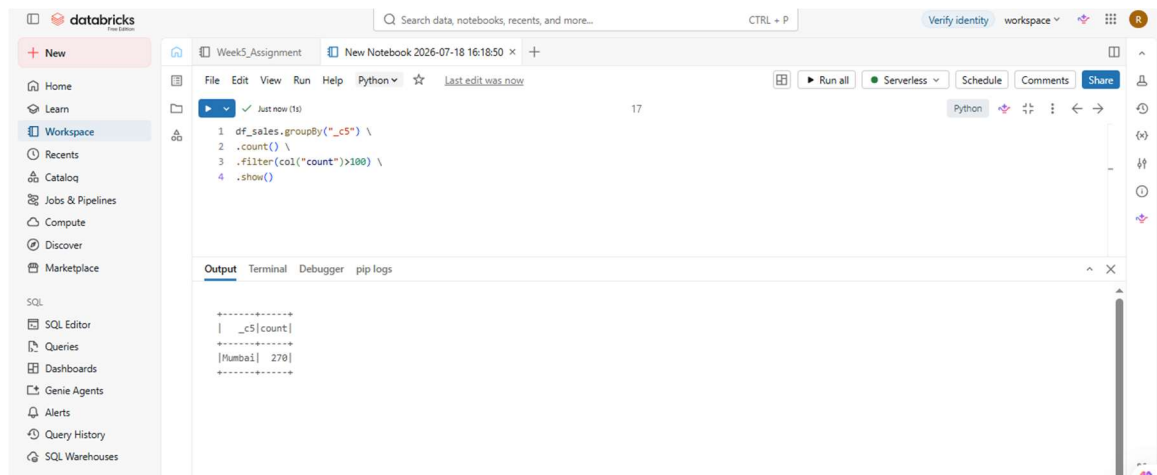
pip logs

	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7	_c8	_c9	_c10	_c11	_c12
user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	status	raw_timestamp	email	username	store_id	
83	2024-03-01	West	Electronics	839.16	Mumbai	41	Premium	Unknown	2024-03-01 11:02:00	user83@example.com	user83	104	
246	2024-02-05	East	Electronics	2180.14	Mumbai	54	Basic	Active	2024-02-05 11:02:00	user24@example.com	user24	187	
37	2024-02-04	East	Electronics		Mumbai	55	Premium	Inactive	2024-02-04 09:35:00	user37@example.com	user37	110	
53	2024-02-01	North	Groceries		Mumbai	55	Basic	Inactive	2024-02-01 12:15:00	user53@example.com	user53	104	
21	2024-02-06	North	Groceries		Mumbai	36	Premium	Active	2024-02-06 16:26:00	user21@example.com	user21	108	
108	2024-01-03	North	Clothing	4813.89	Mumbai	40	Basic	Unknown	2024-01-03 15:04:00	user108@example.com	user108	108	
179	2024-02-12	South	Clothing		Mumbai	42	Basic	Active	2024-02-12 15:22:00	user179@example.com	user179	181	
56	2024-02-19	South	Groceries	1508.42	Mumbai	49	Premium	Active	2024-02-19 15:25:00	user56@example.com	user56	187	

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
df_sales.groupBy("_c5") \
    .count() \
    .filter(col("count")>100) \
    .show()
```

Output:



Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Answer:

Spark DataFrames are immutable, meaning they cannot be changed after creation.

Operations like:

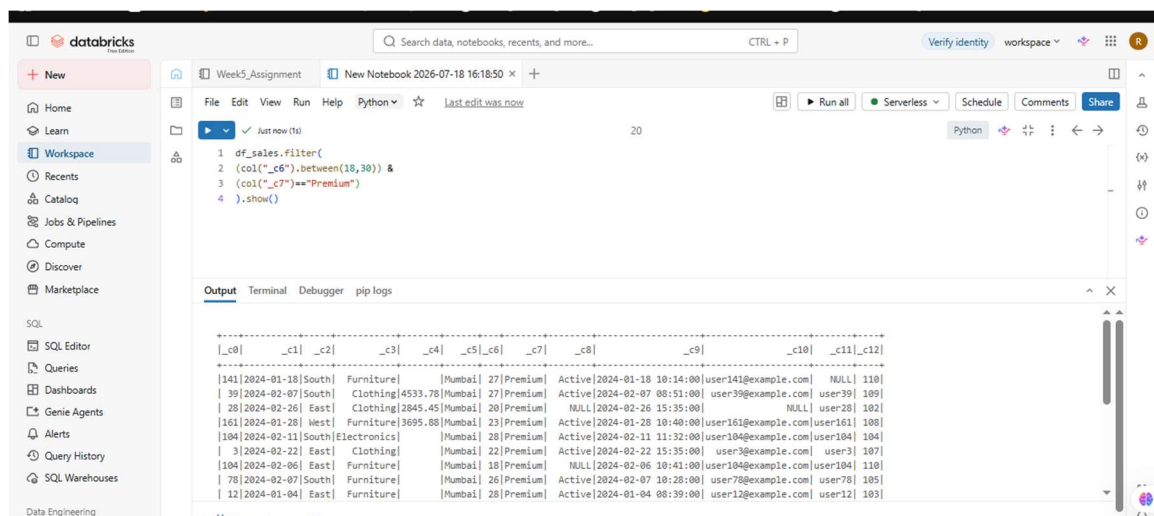
- `drop()`
- `withColumnRenamed()`
- `filter()`

return a new DataFrame rather than modifying the original one.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
df_sales.filter(  
(col("_c6").between(18,30)) &  
(col("_c7")=="Premium")  
) .show()
```

Output:



The screenshot shows the Databricks workspace interface. A notebook titled 'New Notebook 2026-07-18 16:18:50' is open, displaying a Spark SQL query. The query filters the 'df_sales' dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'. The output of the query is displayed in a table format.

_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7	_c8	_c9	_c10	_c11	_c12
141	2024-01-18	South	Furniture		Mumbai	27	Premium	Active	2024-01-18 10:14:00	user141@example.com	NULL	110
39	2024-02-07	South	Clothing	4533.78	Mumbai	27	Premium	Active	2024-02-07 06:51:00	user39@example.com	user39	109
28	2024-02-26	East	Clothing	1045.45	Mumbai	20	Premium	NULL	2024-02-26 15:35:00	NULL	user28	102
161	2024-01-28	West	Furniture	3695.68	Mumbai	23	Premium	Active	2024-01-28 10:40:00	user161@example.com	user161	108
104	2024-02-11	South	Electronics		Mumbai	28	Premium	Active	2024-02-11 11:32:00	user104@example.com	user104	104
3	2024-02-22	East	Clothing		Mumbai	22	Premium	Active	2024-02-22 15:35:00	user3@example.com	user3	107
104	2024-02-06	East	Furniture		Mumbai	18	Premium	NULL	2024-02-06 10:41:00	user104@example.com	user104	110
78	2024-02-07	South	Furniture		Mumbai	26	Premium	Active	2024-02-07 10:28:00	user78@example.com	user78	105
12	2024-01-04	East	Furniture		Mumbai	28	Premium	Active	2024-01-04 08:39:00	user12@example.com	user12	103

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Answer

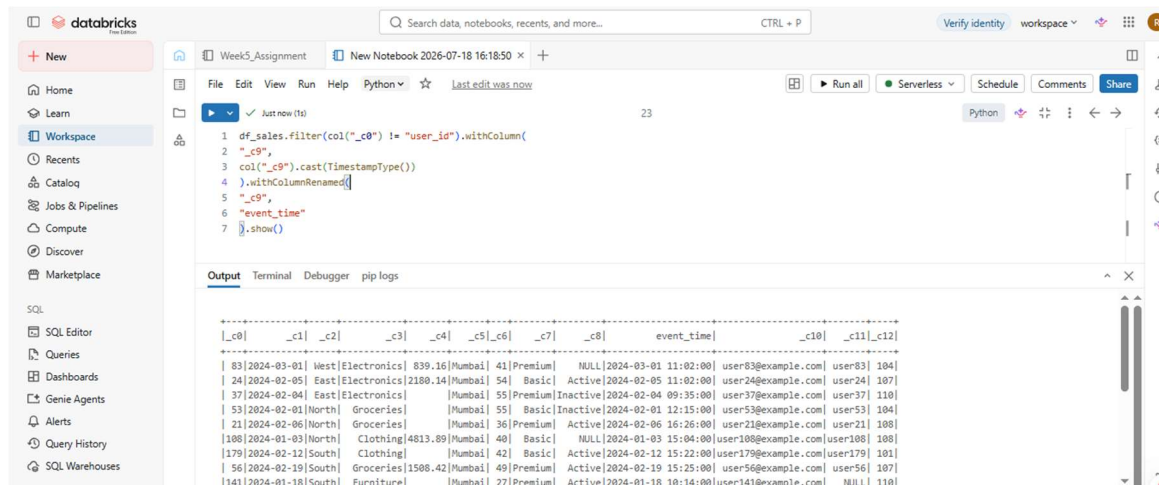
Handling null values before aggregation is important because:

- Null values may be ignored in calculations.
- They can produce inaccurate averages or sums.
- Missing values reduce data quality.
- Cleaning data ensures reliable business insights.

Q10: Write the code to revise a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

```
df_sales.filter(col("_c0") != "user_id").withColumn(
    "_c9",
    col("_c9").cast(TimestampType())
).withColumnRenamed(
    "_c9",
    "event_time"
).show()
```

Output:



The screenshot displays a Databricks workspace with a notebook titled "New Notebook 2026-07-18 16:18:50". The code in the notebook is as follows:

```
1 df_sales.filter(col("_c0") != "user_id").withColumn(
2   "_c9",
3   col("_c9").cast(TimestampType())
4 ).withColumnRenamed(
5   "_c9",
6   "event_time"
7 ).show()
```

The output of the query is shown in a table with 12 columns: `_c0`, `_c1`, `_c2`, `_c3`, `_c4`, `_c5`, `_c6`, `_c7`, `_c8`, `event_time`, `_c10`, and `_c11_c12`. The data is sorted by `event_time`.

_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7	_c8	event_time	_c10	_c11_c12
83	2024-03-01	West	Electronics	839.16	Mumbai	41	Premium	NULL	2024-03-01 11:02:00	user83@example.com	user83 104
24	2024-02-05	East	Electronics	2180.14	Mumbai	54	Basic	Active	2024-02-05 11:02:00	user24@example.com	user24 107
37	2024-02-04	East	Electronics		Mumbai	55	Premium	Inactive	2024-02-04 09:35:00	user37@example.com	user37 110
53	2024-02-01	North	Groceries		Mumbai	55	Basic	Inactive	2024-02-01 12:15:00	user53@example.com	user53 104
21	2024-02-06	North	Groceries		Mumbai	36	Premium	Active	2024-02-06 16:26:00	user21@example.com	user21 108
108	2024-01-03	North	Clothing	4813.89	Mumbai	40	Basic	NULL	2024-01-03 15:04:00	user108@example.com	user108 108
179	2024-02-12	South	Clothing		Mumbai	42	Basic	Active	2024-02-12 15:22:00	user179@example.com	user179 101
56	2024-02-19	South	Groceries	1508.42	Mumbai	49	Premium	Active	2024-02-19 15:25:00	user56@example.com	user56 107
141	2024-01-18	South	Furniture		Mumbai	27	Premium	Active	2024-01-18 10:14:00	user141@example.com	NULL 110

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer:

A shuffle occurs when Spark redistributes data across different partitions during operations such as:

- `groupBy()`
- `join()`
- `distinct()`

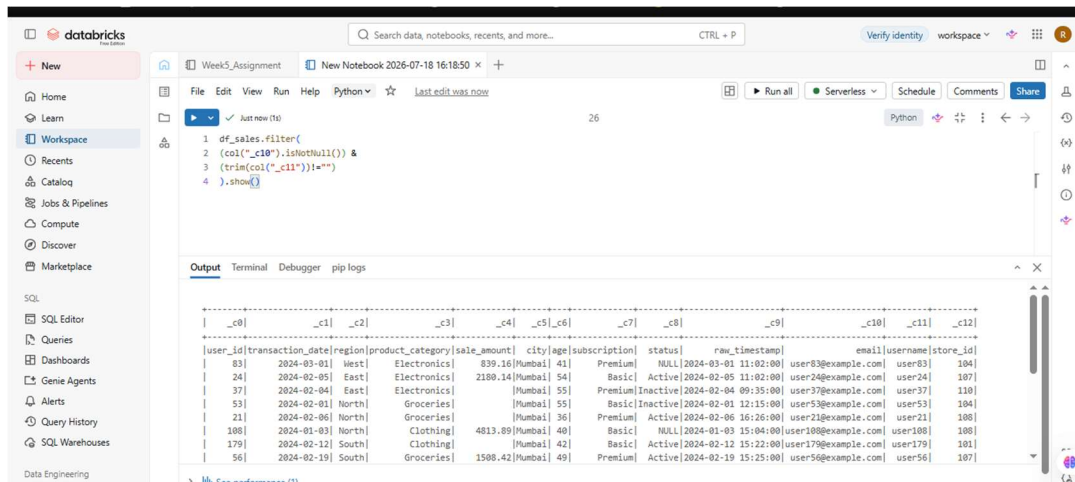
Shuffle is called a wide transformation because each output partition depends on data from multiple input partitions.

It is expensive due to network communication and disk usage.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
df_sales.filter(  
  (col("_c10").isNotNull()) &  
  (trim(col("_c11"))!="")  
)>.show()
```

Output:



The screenshot shows the Databricks interface with a notebook titled 'New Notebook 2026-07-10 16:18:50'. The code in the notebook is:

```
1 df_sales.filter(  
2   (col("_c10").isNotNull()) &  
3   (trim(col("_c11"))!="")  
4 )>.show()
```

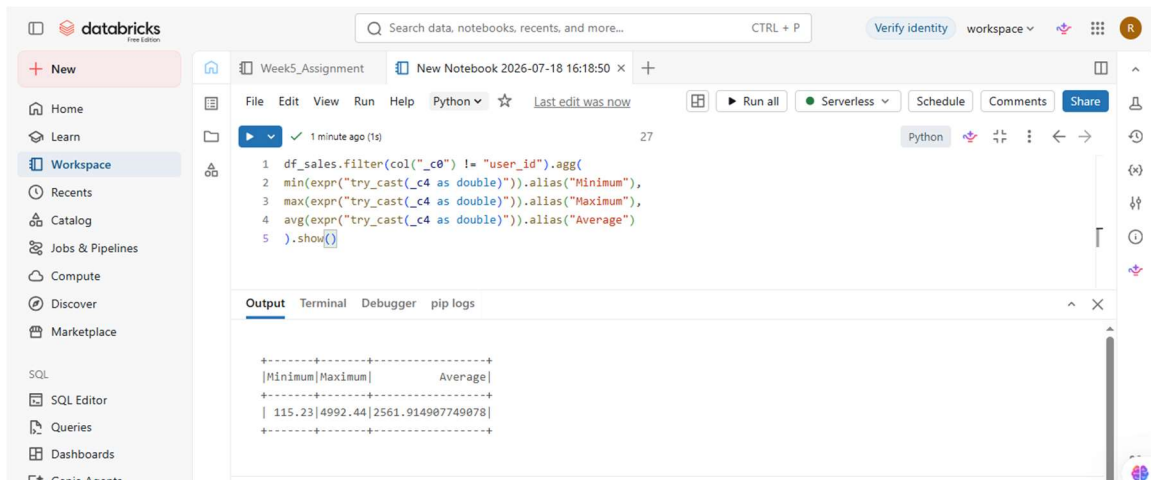
The output of the query is displayed in a table with 12 columns: user_id, transaction_date, region, product_category, sale_amount, city, age, subscription, status, raw_timestamp, email, username, and store_id. The table contains 10 rows of data.

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	status	raw_timestamp	email	username	store_id
83	2024-03-01	West	Electronics	839.16	Mumbai	41	Premium	NULL	2024-03-01 11:02:00	user83@example.com	user83	104
24	2024-02-05	East	Electronics	2180.14	Mumbai	54	Basic	Active	2024-02-05 11:02:00	user24@example.com	user24	107
37	2024-02-04	East	Electronics		Mumbai	55	Premium	Inactive	2024-02-04 09:35:00	user37@example.com	user37	110
53	2024-02-01	North	Groceries		Mumbai	55	Basic	Inactive	2024-02-01 12:15:00	user53@example.com	user53	104
21	2024-02-06	North	Groceries		Mumbai	36	Premium	Active	2024-02-06 16:26:00	user21@example.com	user21	108
108	2024-01-03	North	Clothing	4813.89	Mumbai	40	Basic	NULL	2024-01-03 15:04:00	user108@example.com	user108	108
179	2024-02-12	South	Clothing		Mumbai	42	Basic	Active	2024-02-12 15:22:00	user179@example.com	user179	101
56	2024-02-19	South	Groceries	1588.42	Mumbai	49	Premium	Active	2024-02-19 15:25:00	user56@example.com	user56	107

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

```
df_sales.filter(col("_c0") != "user_id").agg(  
  min(expr("try_cast(_c4 as double)")).alias("Minimum"),  
  max(expr("try_cast(_c4 as double)")).alias("Maximum"),  
  avg(expr("try_cast(_c4 as double)")).alias("Average")  
)>.show()
```

Output:



Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Ans:

When source data contains inconsistent date formats, inferSchema=True may:

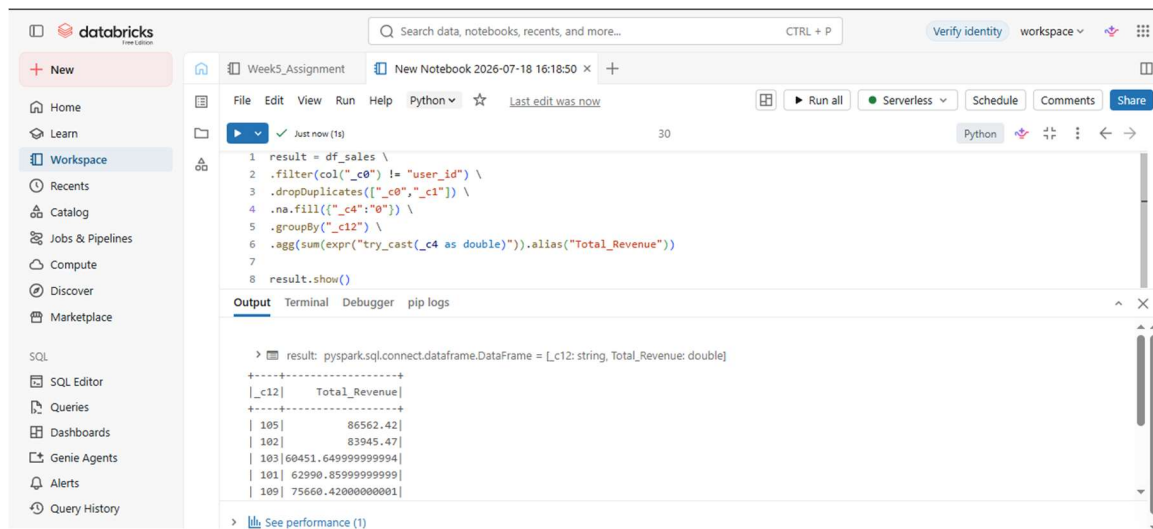
- Infer the wrong data type.
- Treat dates as strings.
- Produce null values during parsing.
- Cause incorrect query results.

Using an explicit schema is safer for production datasets.

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by store_id to calculate total revenue.

Output:



The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. The main area displays a notebook titled 'New Notebook 2026-07-18 16:18:50'. The code in the notebook is as follows:

```
1 result = df_sales \
2   .filter(col("c0") != "user_id") \
3   .dropDuplicates(["c0", "c1"]) \
4   .na.fill({"c4": "0"}) \
5   .groupBy("c12") \
6   .agg(sum(expr("try_cast(_c4 as double)").alias("Total_Revenue")))
7
8 result.show()
```

Below the code, the 'Output' tab is active, showing the execution result. It displays a table with two columns: 'c12' and 'Total_Revenue'.

c12	Total_Revenue
105	86562.42
102	83945.47
103	60451.649999999994
101	62990.859999999999
109	75660.420000000001

Brief Insights:

- Spark is much faster than MapReduce because it performs in-memory computation.
- Spark DataFrames are immutable, so every transformation creates a new DataFrame.
- Data cleaning (removing duplicates and handling null values) improves data quality.
- Filtering data before aggregation reduces processing time.
- groupBy() and aggregation functions provide useful business insights.
- Shuffle operations are expensive because they transfer data across partitions.
- Explicit schemas are more reliable than inferSchema=True when data is inconsistent.
- An end-to-end Spark pipeline combines data cleaning → transformation → aggregation → analysis efficiently.

Conclusion:

In this Week-5 assignment, Apache Spark DataFrame operations were successfully used to perform **data cleaning, transformation, and aggregation**. Duplicate records were removed, null values were handled, and filtering operations were applied to improve data quality. Aggregation functions such as **count()**, **sum()**, **avg()**, **min()**, and **max()** were used to generate meaningful insights, while **groupBy()** enabled analysis of data by categories such as product and store.

The assignment also demonstrated important Spark concepts, including **DataFrame immutability, schema modification, in-memory computing, and shuffle operations**. Compared to traditional MapReduce, Spark provides much faster processing by keeping intermediate data in memory, making it well suited for iterative and large-scale data processing tasks.

Overall, this assignment showed how Spark can efficiently build a complete data processing pipeline—from **data cleaning and transformation to aggregation and analysis**—making it a powerful framework for modern big data applications.